

Text Generation Inference with LLMs —

১. Text Generation Inference কী?

LLM (Large Language Model) সবচেয়ে বেশি ব্যবহার হয় **text generate** করার জন্য — যেমন:

- Chatbot (ChatGPT)
- Story generation
- Code generation
- Translation
- Summarization

এখানে **Inference** মানে হলো:

Train করা Model ব্যবহার করে নতুন **text** তৈরি করা।

সহজভাবে

- Training → Model শেখে
- Inference → Model উত্তর দেয় / text generate করে

LLM একবার train হয়ে গেলে, inference সময় model একটা **prompt** থেকে ধাপে ধাপে **word/token predict** করে।

২. LLM কিভাবে Text Generate করে?

LLM text generate করে **token by token**।

উদাহরণ:

Input Prompt:

The capital of France is

Model ভাবে:

- Next token probability:
 - Paris → 0.92
 - London → 0.03

- Rome → 0.02

Model সবচেয়ে বেশি probability যেটার → সেটাই নেয় → **Paris**

তারপর আবার next token predict করে।

Core Concept

LLM uses:

- Billions of parameters
- Probability distribution
- Sequential prediction

এই কারণেই LLM coherent এবং human-like text তৈরি করতে পারে।

৩. Attention Mechanism — LLM এর Brain

LLM এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো **Attention Mechanism**।

Attention কী করে?

Sentence:

The capital of France is ...

সব word সমান important না।

Important words:

- capital
- France

Model এই word গুলোতে বেশি focus করে → next word predict করে → **Paris**

এই focus করার process = **Attention**

কেন **Attention** এত **Powerful**?

Attention model কে সাহায্য করে:

- Context বুঝতে
- Relevant word detect করতে
- Long sentence handle করতে
- Meaningful response generate করতে

এটা ছাড়া LLM **context** বুঝতে পারতো না।

8. Modern LLM vs Old Language Model

আগের Model (n-gram, RNN, LSTM):

- Short context
- Memory problem
- Long sentence ভুলে যেত

Transformer + Attention (Modern LLM):

- Long context handle
- Parallel computation
- Better understanding
- Coherent text

BERT, GPT-2 থেকে শুরু → আজকের GPT, LLaMA, Mistral → huge improvement।

৫. Context Length (Model এর Attention Span)

এখন আসি **Context Length** এ।

Context Length কী?

Model একবারে কত **token** মনে রাখতে পারে → সেটাই context length।

Think like:

Model working memory size

Example:

- 2K tokens → ছোট context
- 8K tokens → মাঝারি
- 32K / 128K → বড় context

Context Length কেন Limit থাকে?

কারণ:

1. Model architecture
2. GPU memory
3. Computational cost
4. Input complexity

Context যত বড় → computation cost তত বেশি।

Context Length কী প্রভাব ফেলে?

Small Context	Large Context
Short memory	Long memory
Long doc বুঝতে পারে না	Whole document বুঝে
Context lose হয়	Better reasoning
Cheaper	Expensive

৬. Token কী?

Token মানে:

- Word
- Word part
- Character piece

Example:

unbelievable → un + believe + able

LLM word না → **token** দিয়ে কাজ করে।

৭. LLM Inference Pipeline

Text generation সময় model এই process follow করে:

1. Prompt input নেয়
2. Tokenize করে
3. Attention দিয়ে context analyze করে
4. Next token probability calculate করে
5. Token select করে (greedy / sampling)
6. Repeat until end token

এটাকে বলে **autoregressive generation**

৮. Extra Important Concepts (Deep Level)

Temperature

Randomness control করে:

- Low → deterministic
- High → creative

Top-K / Top-P

Token selection control করে।

Greedy vs Sampling

- Greedy → best probability
- Sampling → creative text

The Art of Prompting —

৯. Prompting কী?

LLM কে আমরা যে ইনপুট দিই (instruction / question / context) সেটাকে বলা হয় **Prompt**।
এই prompt এমনভাবে লিখতে হয় যাতে model বুঝতে পারে আমরা কী ধরনের output চাই।

এটাকেই বলা হয় **Prompting**।

সহজভাবে:

- Prompt = Model কে নির্দেশনা দেওয়া
- ভালো Prompt = ভালো Output
- খারাপ Prompt = ভুল বা দুর্বল Output

কারণ LLM আসলে শুধু **next token predict** করে। তাই তুমি কীভাবে লিখছো, কোন শব্দ ব্যবহার করছো — সেটা খুব গুরুত্বপূর্ণ।

২. কেন Prompt Design গুরুত্বপূর্ণ?

LLM কাজ করে:

- Input token analyze করে
- গুরুত্বপূর্ণ token detect করে (attention)
- তারপর next token predict করে

তাই যদি prompt পরিষ্কার, structured, specific হয়:

- Model সহজে বুঝবে
- Output বেশি accurate হবে
- Hallucination কম হবে
- Reasoning improve হবে

Example

Bad prompt:

Explain AI

Good prompt:

Explain Artificial Intelligence in simple terms with real-life examples and short bullet points

দেখা যাচ্ছে, second prompt বেশি clear, তাই output ভালো হবে।

৩. Two-Phase Inference Process

LLM text generation দুইটি phase এ হয়:

1. Prefill Phase
2. Decode Phase

এই দুইটা phase একসাথে কাজ করে — যেমন assembly line।

Prefill Phase —

Prefill হলো preparation stage।

এখানে model পুরো **input prompt process** করে।

এই phase এ তিনটি main step হয়:

Step 1: Tokenization

Input text → Token এ convert হয়।

Example:

I love machine learning

→ [I] [love] [machine] [learning]

Model শুধু token বুঝে, word না।

Step 2: Embedding Conversion

Token → Number vector এ convert হয়।

কারণ neural network শুধু সংখ্যা বুঝে।

Embedding represent করে:

- Meaning
- Semantic relation
- Context hint

Example:

king – man + woman $\approx$ queen (vector space logic)

Step 3: Initial Processing (Context Understanding)

Embedding → Transformer network → Attention → Context representation তৈরি হয়।

এখানে model:

- পুরো prompt পড়ে
- relation বুঝে
- semantic structure তৈরি করে

এই phase heavy computation লাগে কারণ সব **input token** একসাথে **process** হয়।

Think like:

Model পুরো paragraph পড়ে তারপর লেখা শুরু করে।

Decode Phase —

Prefill শেষে শুরু হয় **Decode Phase**।

এখানেই actual text generation হয়।

Model একটা করে **token generate** করে — এটাকে বলে autoregressive generation।

Decode Phase Steps

প্রতিটি নতুন token generate করার সময় model নিচের কাজগুলো করে:

১. Attention Computation

Model আগের সব token এর দিকে তাকায় (prompt + generated tokens) → context বুঝে।

২. Probability Calculation

সব possible next token এর probability calculate করে।

Example:

Paris → 0.92

London → 0.04

Rome → 0.02

৩. Token Selection

Sampling strategy অনুযায়ী next token select করে।

8. Continuation Check

Model decide করে:

- Continue করবে?
- নাকি stop (EOS token)?

এই process repeat হয় → complete sentence তৈরি হয়।

Prefill vs Decode — Core Difference

Prefill	Decode
Input process করে	Text generate করে
Compute heavy	Memory heavy
All tokens at once	One token at a time
Context build	Output create

Sampling Strategies — Generation Control

LLM কিভাবে token select করবে — সেটা control করা যায়।

এটাকে বলে **Sampling Strategy**।

১. Greedy Decoding

সবচেয়ে বেশি probability token নেয়।

Pros:

- Deterministic
- Fast
- Stable

Cons:

- Creative না
- Repetitive হতে পারে

২. Temperature

Randomness control করে।

Low temperature (0.2):

- Safe
- Deterministic
- Same output

High temperature (0.9):

- Creative
- Diverse
- Unpredictable

৩. Top-K Sampling

শুধু Top K tokens থেকে select করে।

Example:

$K = 5 \rightarrow$ only top 5 candidate

Noise কমায়।

৪. Top-P (Nucleus Sampling)

Probability cumulative threshold set করে।

Example:

$P = 0.9 \rightarrow$ smallest token set যার sum ≥ 0.9

Balanced randomness।

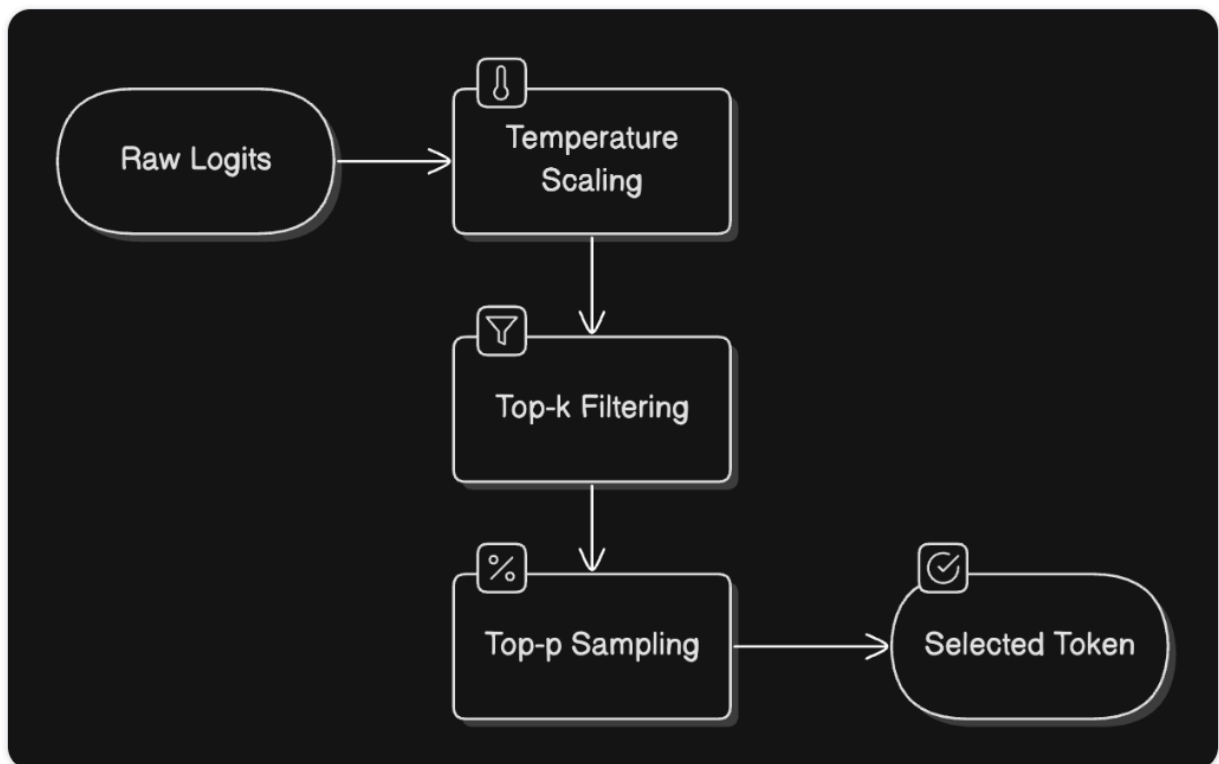
৫. Beam Search

Multiple candidate sequence explore করে → best sentence choose করে।

Used in:

- Translation
- Summarization

Understanding Token Selection —



১. Logits থেকে Token Selection — মূল ধারণা

LLM যখন next token choose করে, তখন প্রথমে model **logits** তৈরি করে।

Logits কী?

Logits হলো:

- Model এর raw score / unnormalized probability
- প্রতিটি vocabulary token এর জন্য একটি সংখ্যা
- এখনও probability না

উদাহরণ (Raw logits):

Token	Logit
-------	-------

Paris	5.2
-------	-----

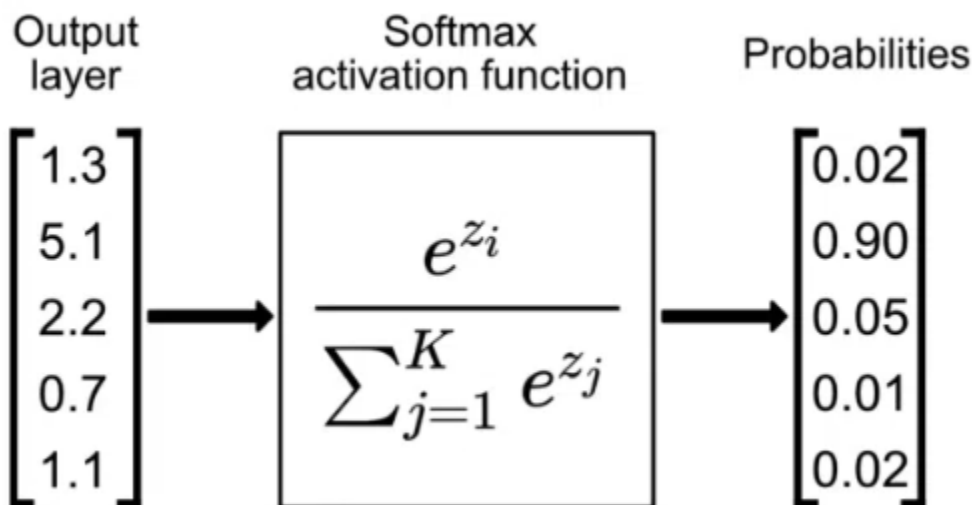
London	2.1
--------	-----

Rome	1.3
------	-----

এই logits সরাসরি probability না। এগুলোকে **softmax function** দিয়ে probability বানানো হয়।

Softmax formula:

$$p_i = \exp(\text{logit}_i) / \sum(\exp(\text{logits}))$$



এরপর পাওয়া যায় probability distribution।

২. Temperature — Randomness Control

Temperature logits কে scale করে randomness control করে।

Formula:

$\text{new_logit} = \text{logit} / \text{temperature}$

Temperature Effect

Temperature	Behaviour
< 0.5	Deterministic, safe
= 1	Normal
> 1	Random, creative

Low temperature:

- Same output
- Focused
- Less diversity

High temperature:

- More creative
- Risky
- More variation

৩. Top-p (Nucleus Sampling)

সব token consider না করে শুধু **highest probability tokens** নেওয়া হয় যাদের cumulative probability $\geq p$

Example:

Token	Probability
-------	-------------

Paris 0.50

London 0.25

Rome 0.10

Berlin 0.08

Madrid 0.04

If $p = 0.90 \rightarrow$ select:

Paris + London + Rome + Berlin = 0.93

এই subset থেকে sampling হয়।

Benefit:

- Noise reduce
- Balanced randomness
- Natural text

8. Top-k Sampling

Top k most probable tokens থেকে select করা হয়।

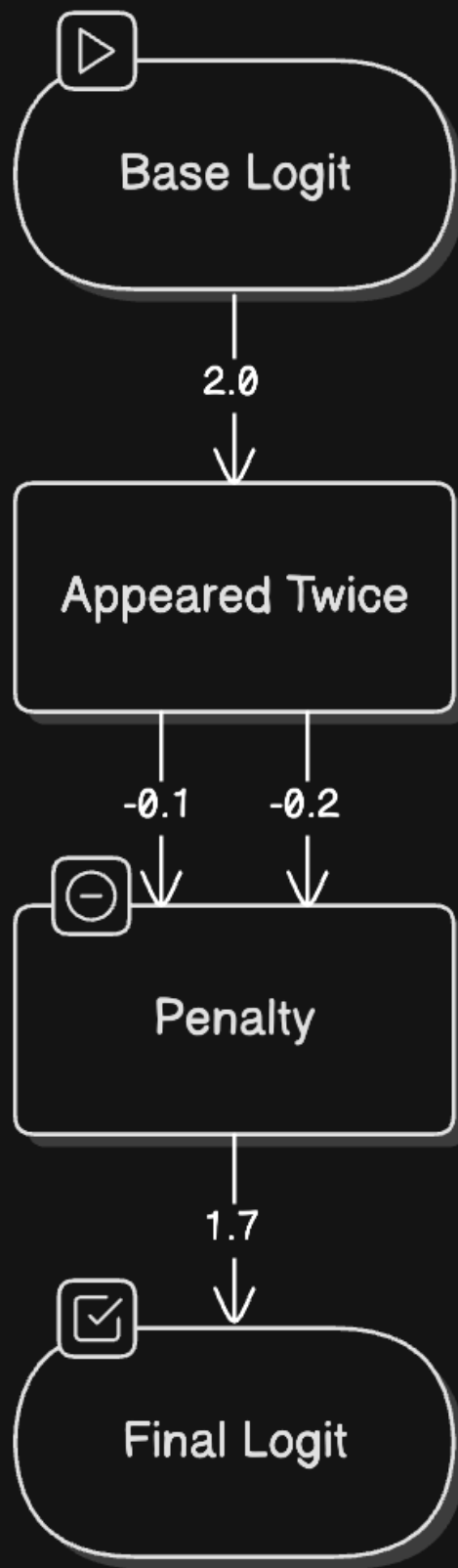
Example:

$k = 3 \rightarrow \{\text{Paris, London, Rome}\}$

Simple but less adaptive than top-p

Managing Repetition — Repetition Control

LLM অনেক সময় একই শব্দ বারবার repeat করে। এটাকে control করতে penalty ব্যবহার করা হয়।



১. Presence Penalty

যে token একবার এসেছে → penalty apply হয় (frequency matter না)।

Effect:

- Same word reuse কমে
- Topic shift encourage করে
- More diverse output

২. Frequency Penalty

Token যতবার repeat হয় → penalty তত বাড়ে।

Effect:

- Repetition strongly reduce
- Looping stop করে
- Vocabulary diversify করে

এই penalties **sampling** এর আগে **logits adjust** করে।

Controlling Generation Length — Output Size Control

LLM কত বড় text generate করবে — সেটা control করা যায়।

১. Token Limit

Set করা যায়:

- Min tokens
- Max tokens

Example:

max_tokens = 100 → model 100 token এর বেশি generate করবে না।

২. Stop Sequence

Specific pattern দিলে generation stop হয়।

Example:

Stop = "\n\n"

Used in:

- Chat
- Structured output
- Paragraph control

৩. EOS Token (End of Sequence)

Model নিজে decide করে generation শেষ করবে।

Used in:

- Natural completion
- Sentence finish

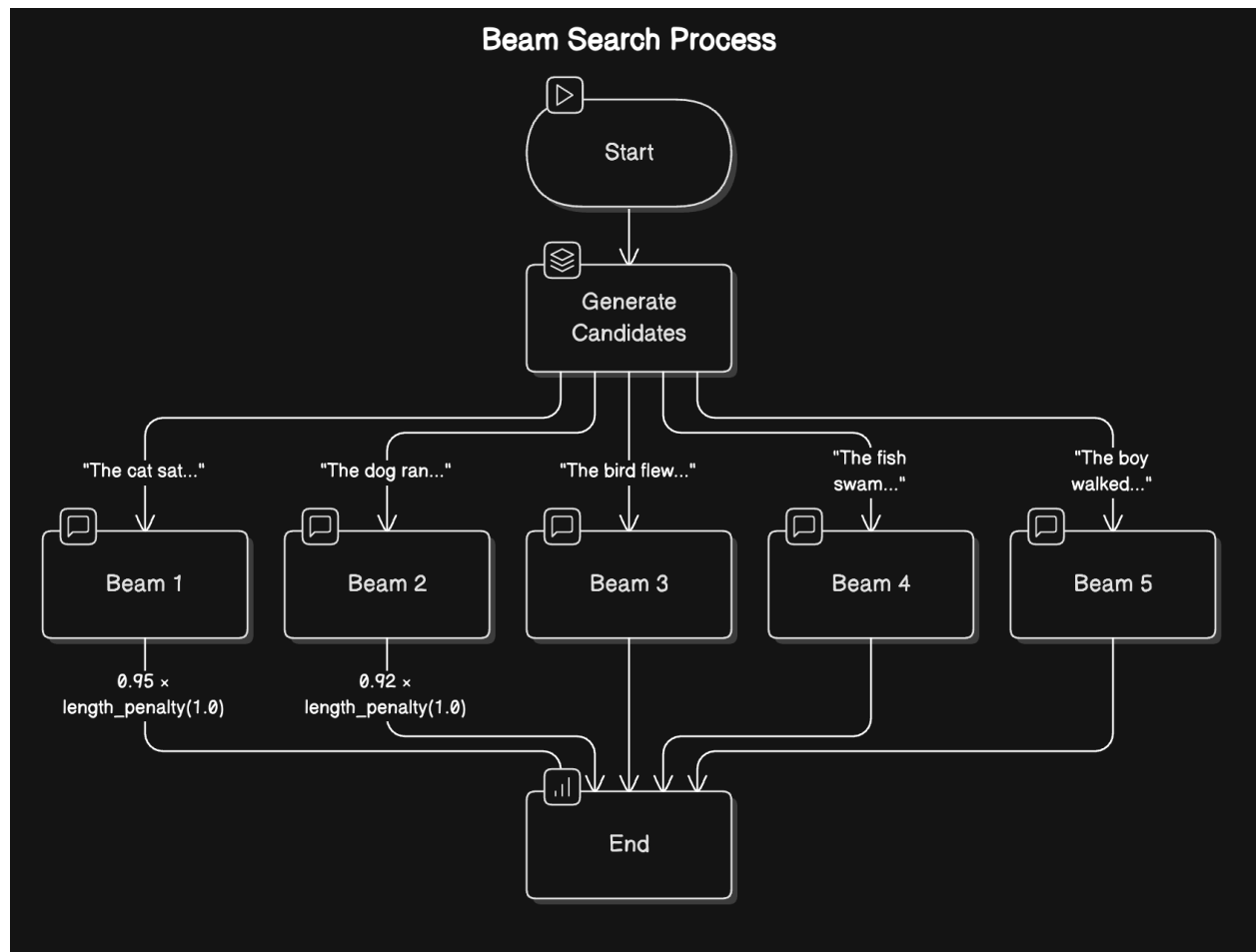
Beam Search — Deep Look Ahead Generation

Greedy বা sampling একসময় একটাই token select করে।

কিন্তু **Beam Search multiple sequence** একসাথে **explore** করে।

Think like:

Chess player কয়েকটা move আগে চিন্তা করছে।



Beam Search Working

Step-by-step:

1. Start with prompt
2. Maintain multiple candidate sequences (beam width = 5–10)
3. Each sequence → next token probability compute
4. Keep best scoring sequences
5. Repeat until stop condition
6. Highest probability sequence choose

Example (Beam width = 2)

Step 1:

"I love"

→ {machine, deep}

Step 2:

"I love machine" → learning

"I love deep" → learning

Step 3:

Score compare → best sentence choose

Beam Search Advantages

- More coherent
- Grammatically correct
- Better long sentence

Disadvantages

- Compute heavy
- Less creative
- Deterministic

Used in:

- Machine translation
- Summarization
- Speech recognition

Token Selection Full Pipeline

Logits → Temperature scaling → Repetition penalty → Top-k / Top-p filtering → Softmax → Sampling → Token selected

Deep Insight

Sampling methods control:

- Creativity
- Diversity
- Stability
- Coherence

Penalty controls:

- Repetition
- Vocabulary richness

Length control manages:

- Output size
- Structure

Beam search optimizes:

- Global sentence probability

Practical Challenges and Optimization in LLM Inference —

LLM শুধু theoretical model না। বাস্তবে deploy করতে গেলে speed, memory, cost, scalability — এই সব বড় challenge হয়ে দাঁড়ায়। তাই inference optimization খুব গুরুত্বপূর্ণ।

১. Key Performance Metrics

LLM system evaluate করার জন্য ৪টি সবচেয়ে গুরুত্বপূর্ণ metric আছে।

১. Time to First Token (TTFT)

Definition:

User prompt দেওয়ার পর প্রথম **token generate** হতে যত সময় লাগে।

Mainly depends on:

- Prefill phase (input processing)
- Context length
- Model size
- GPU speed

Impact:

- User experience directly affect করে
- Chat system এ latency feel হয়

Optimization ideas:

- Shorter prompt
- KV cache reuse
- Faster tokenizer
- Quantized model

২. Time Per Output Token (TPOT)

Definition:

একটা token generate করতে average সময়।

Depends on:

- Decode phase speed
- GPU compute
- Model size
- KV cache efficiency

Impact:

- Overall generation speed
- Long output → total time = TPOT × token count

Optimization:

- KV cache
- Flash attention
- Speculative decoding
- Efficient batching

৩. Throughput

Definition:

একই সময়ে system কত request handle করতে পারে।

Depends on:

- GPU memory
- Batch size
- Parallelism
- Model quantization

Impact:

- Scalability
- Cost efficiency
- Production performance

Optimization:

- Dynamic batching
- Tensor parallelism
- Model sharding
- Request scheduling

8. VRAM Usage (GPU Memory)

Definition:

Model run করতে কত GPU memory লাগে।

VRAM use হয়:

- Model weights
- KV cache
- Activations
- Attention matrices

Real-world constraint:

অনেক সময় VRAM সবচেয়ে বড় bottleneck।

Optimization:

- Quantization (INT8, INT4)
- Offloading (CPU + GPU)
- KV cache compression
- Smaller model

২. Context Length Challenge

Context length বাড়লে model বেশি information বুঝতে পারে, কিন্তু cost দ্রুত বাড়ে।

Memory Growth

Transformer attention complexity:

Memory $\propto n^2$

(n = context length)

Example:

4K $\rightarrow$ 16K $\rightarrow$ 64K $\rightarrow$ memory explode করে

কারণ:

Attention matrix size = $n \times n$

Speed Impact

Processing time $\propto n$

Context যত বড়:

- Prefill slow
- Attention heavy
- Decode latency বাড়ে

Real Trade-off

Small Context	Large Context
Fast	Slow
Cheap	Expensive
Limited memory	Better reasoning
Short input	Long doc understanding

Recent models 1M tokens handle করতে পারে, কিন্তু inference অনেক slow হয়। তাই practical system এ **optimal context length choose** করতে হয়।

৩. KV Cache Optimization — Core Speed Trick

LLM decode phase এ repeatedly same attention calculation হয়। KV cache এই repeated computation avoid করে।

KV Cache কী?

Transformer attention uses:

- Key (K)
- Value (V)

KV cache store করে:

- Previous tokens এর K, V
- Future decode step এ reuse করে

Without KV Cache

প্রতিটি নতুন token generate করার সময়:

- পুরো sequence আবার compute হয়
- $O(n^2)$ computation

Very slow.

With KV Cache

শুধু নতুন token compute হয়:

- Old tokens reuse
- Computation $\approx O(n)$

Result:

- Decode speed huge increase
- Long context practical হয়
- Latency কমে

Trade-off

KV cache:

- Speed বাড়ায়
- But extra VRAM লাগে

Real systems:

Speed benefit $\gg$ Memory cost

8. Practical Optimization Techniques

Production LLM inference এ নিচের techniques use হয়:

১. Quantization

FP16 → INT8 → INT4

Benefit:

- VRAM reduce
- Throughput increase
- Faster compute

Cost:

- Slight accuracy drop

২. Flash Attention

Memory efficient attention algorithm।

Benefit:

- Faster
- Less VRAM
- Large context possible

৩. Speculative Decoding

Small model predict করে → large model verify করে।

Benefit:

- Faster generation
- Latency reduce

৪. Batching

Multiple requests একসাথে process।

Benefit:

- GPU utilization maximize
- Throughput increase

৫. Tensor / Pipeline Parallelism

Model multiple GPU তে distribute করা।

Benefit:

- Large model run possible
- Scaling improve

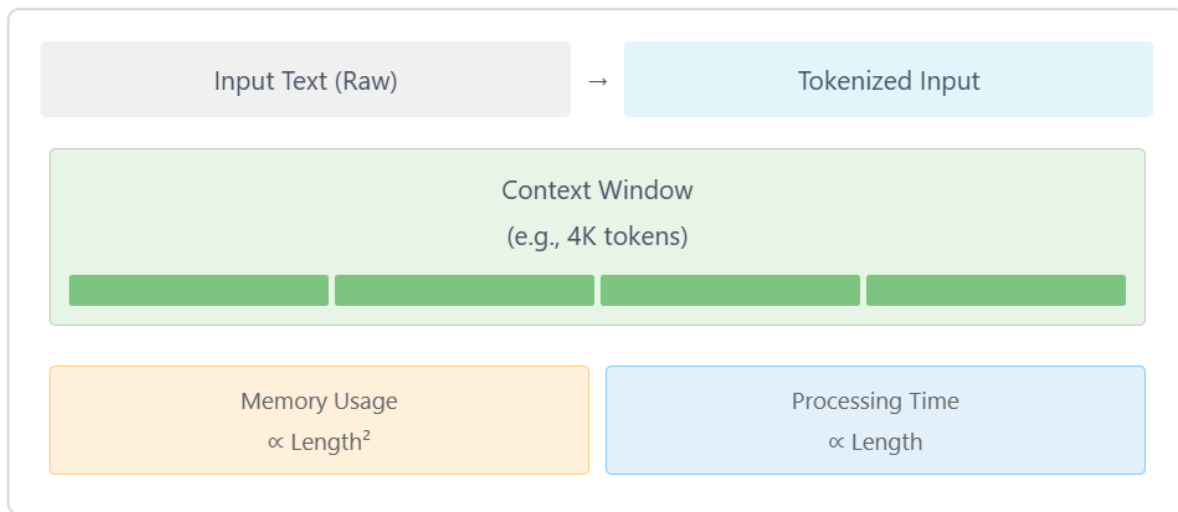
৫. Full Inference Bottleneck Map

Major bottlenecks:

1. Prefill latency (TTFT)
2. Decode speed (TPOT)
3. Memory (VRAM)
4. Context scaling
5. Attention complexity

Optimization target:

- Reduce latency
- Increase throughput
- Reduce memory
- Maintain quality



📌. Conceptual Summary

LLM inference depends on:

- Attention → Context understanding
- Prefill → Input processing
- Decode → Token generation
- Sampling → Output control
- Metrics → Performance measure
- KV Cache → Speed optimization
- Quantization → Memory optimization